

Lecture 02 - Image Filtering

Prof. André Gustavo Hochuli

gustavo.hochuli@pucpr.br

aghochuli@ppgia.pucpr.br

Topics

- Discussion of Practice 01
- Image Filtering
 - Convolution
 - Smoothing, Shapening and Edge Filters
- Practice

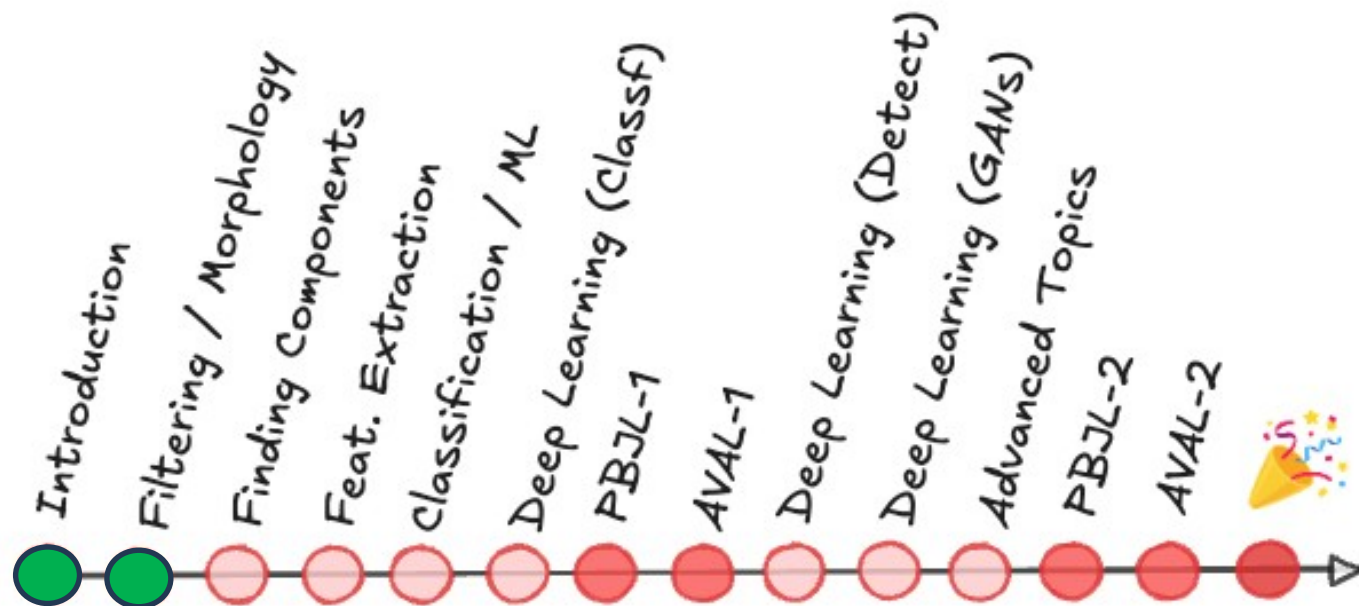
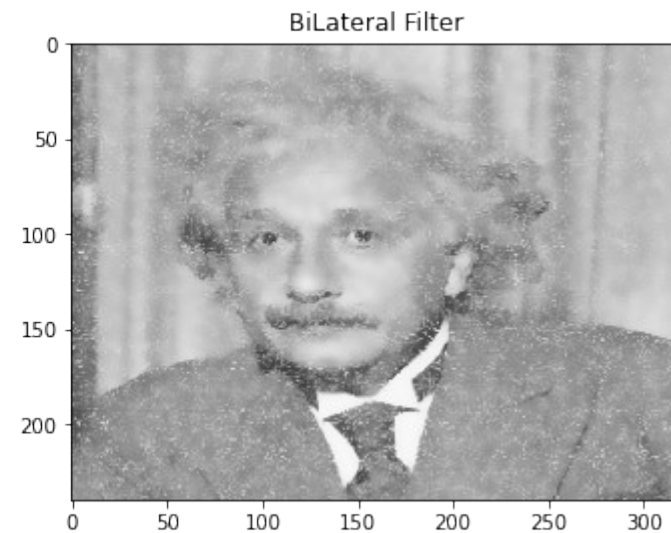
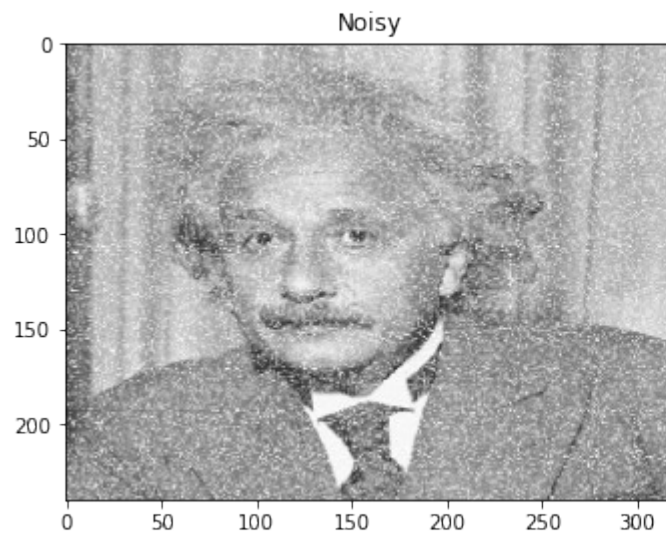


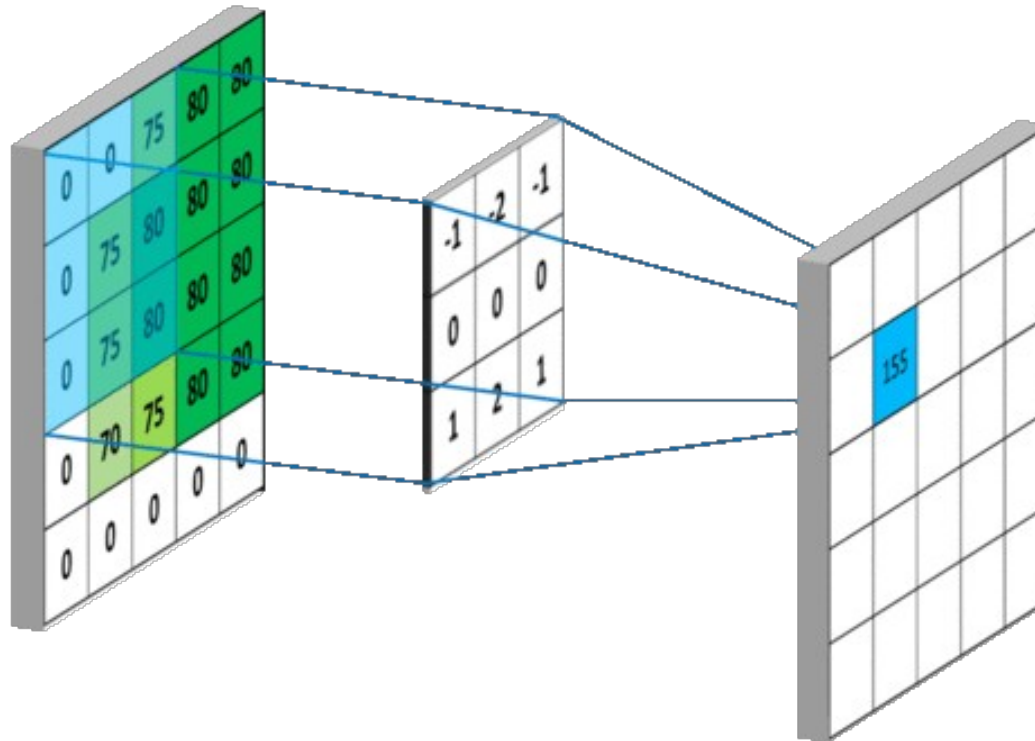
Image Filtering

- Image Enhancement
- Noise Reduction
- Smoothing, Sharpening and Edge Filter
- Convolutional-Based Operations



Convolution

- Slides a kernel (a.k.a convolution filter) in the entire image
- Transforming the pixel in the center of the kernel by the weights of its neighbors



Convolution

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

$$\begin{aligned} &7 \times 1 + 4 \times 1 + 3 \times 1 + \\ &2 \times 0 + 5 \times 0 + 3 \times 0 + \\ &3 \times -1 + 3 \times -1 + 2 \times -1 \\ &= 6 \end{aligned}$$

Spatial Frequency

- Spatial frequency: rate of intensity variation across space
 - Low Frequency → Slow Intensity Variations
 - Smooth or homogeneous regions : [100, 102, 98, 103, 99]
- Small changes between neighboring pixels → low frequency
 - High Frequency: Rapid Intensity Variations
 - Edges, fine details, textures, and often noise: [100, 145, 73, 132, 88]

Filter	Preserves	Attenuates	Typical Effect	Examples
Low-Pass	Low frequencies	High frequencies	Smoothing / reduction of noise and fine details	Mean, Gaussian
High-Pass	High frequencies	Low frequencies	Enhancement of edges and fine details	Laplacian, Sobel*

Spatial Frequency

Input Image

LOW FREQUENCY (Smooth regions)

Areas with gradual intensity changes, such as the sky and clouds.



Example pixel values
(5×5 matrix)

181	182	183	184	185
180	182	183	184	185
181	182	184	185	186
182	183	184	185	186
182	183	185	186	187

Neighboring pixels change only slightly → slow spatial variation → **low frequency**.

HIGH FREQUENCY (Detailed regions)

Areas with rapid intensity changes, such as tree edges and textures.



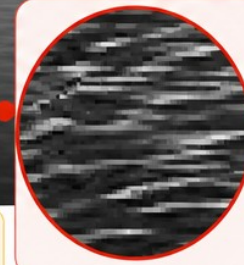
Example pixel values
(5×5 matrix)

42	48	176	189	51
45	172	185	55	47
168	181	49	43	174
179	53	46	169	188
51	44	171	184	57

Neighboring pixels change strongly → rapid spatial variation → **high frequency**.

HIGH FREQUENCY (Detailed regions)

Areas with rapid intensity changes, such as water ripples and reflections.



Example pixel values
(5×5 matrix)

96	112	157	109	95
101	158	166	112	98
160	170	108	97	155
173	111	100	159	168
98	95	156	165	110

Neighboring pixels change strongly → rapid spatial variation → **high frequency**.

 LOW FREQUENCY (Smooth)

→ Gradual intensity changes, large smooth areas.

 HIGH FREQUENCY (Detailed)

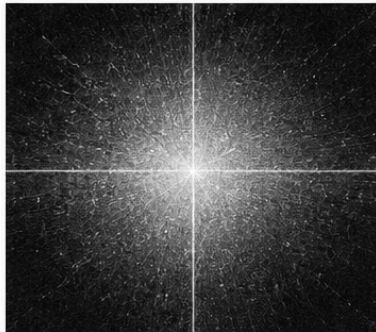
→ Rapid intensity changes, edges, fine details, textures.

Low- and High-Pass Filters

INPUT IMAGE



FREQUENCY SPECTRUM (LOG SCALE)

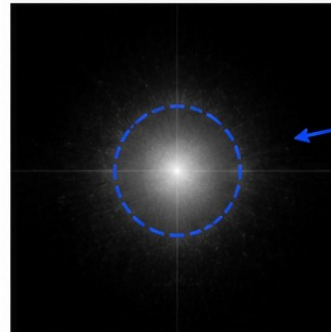


The center contains low frequencies (smooth variations).
The outer regions contain high frequencies (edges, fine details, noise).

LOW-PASS (LOW FREQUENCY)



FREQUENCY SPECTRUM (LOG SCALE)



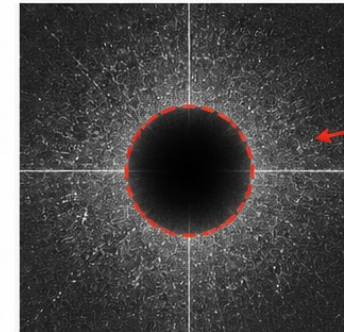
Only low frequencies are kept (center region)

- High frequencies are removed.
- Image appears smooth/blurred.
- Noise and fine details are suppressed.

HIGH-PASS (HIGH FREQUENCY)



FREQUENCY SPECTRUM (LOG SCALE)



Low frequencies are removed (center region)

- Low frequencies are removed.
- Edges and fine details are emphasized.
- Background looks dark / flat.

Smothing

Smoothing Filters

- Goal: Reduce rapid intensity variations and attenuate image noise
- Main Effect: Produce smoother images by suppressing high-frequency components



Why Smoothing?

- Many noises appear as rapid intensity changes.
- Smoothing suppresses high-frequency components.
- Useful as a preprocessing step for many tasks (e.g., segmentation, detection).

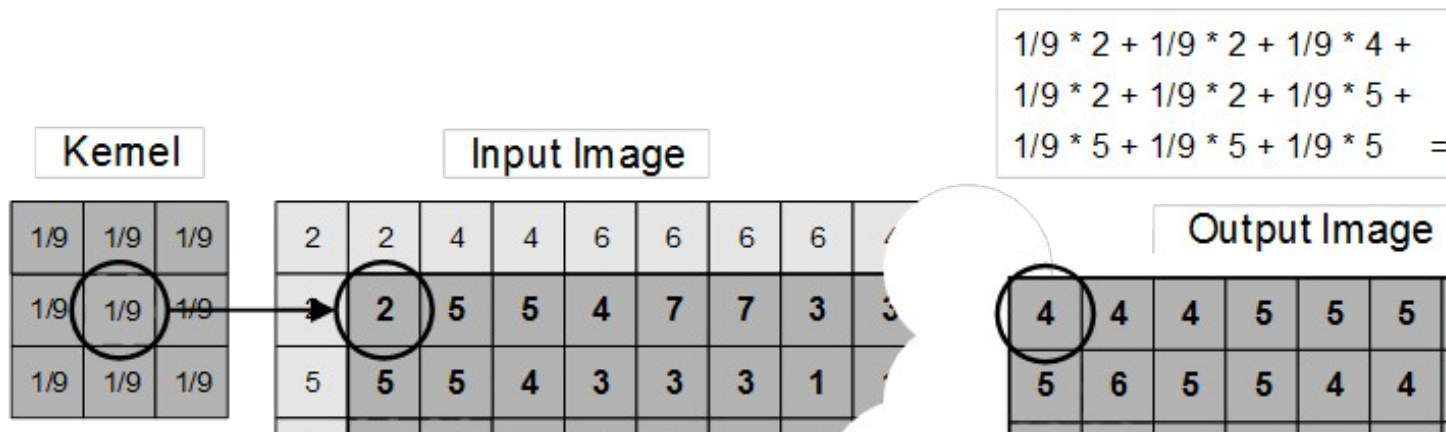


Takeaway

All these filters reduce high-frequency variations (noise), but they differ in how they treat edges and details. Mean and Gaussian are linear and simple; Median and Bilateral are non-linear and better preserve important structures.

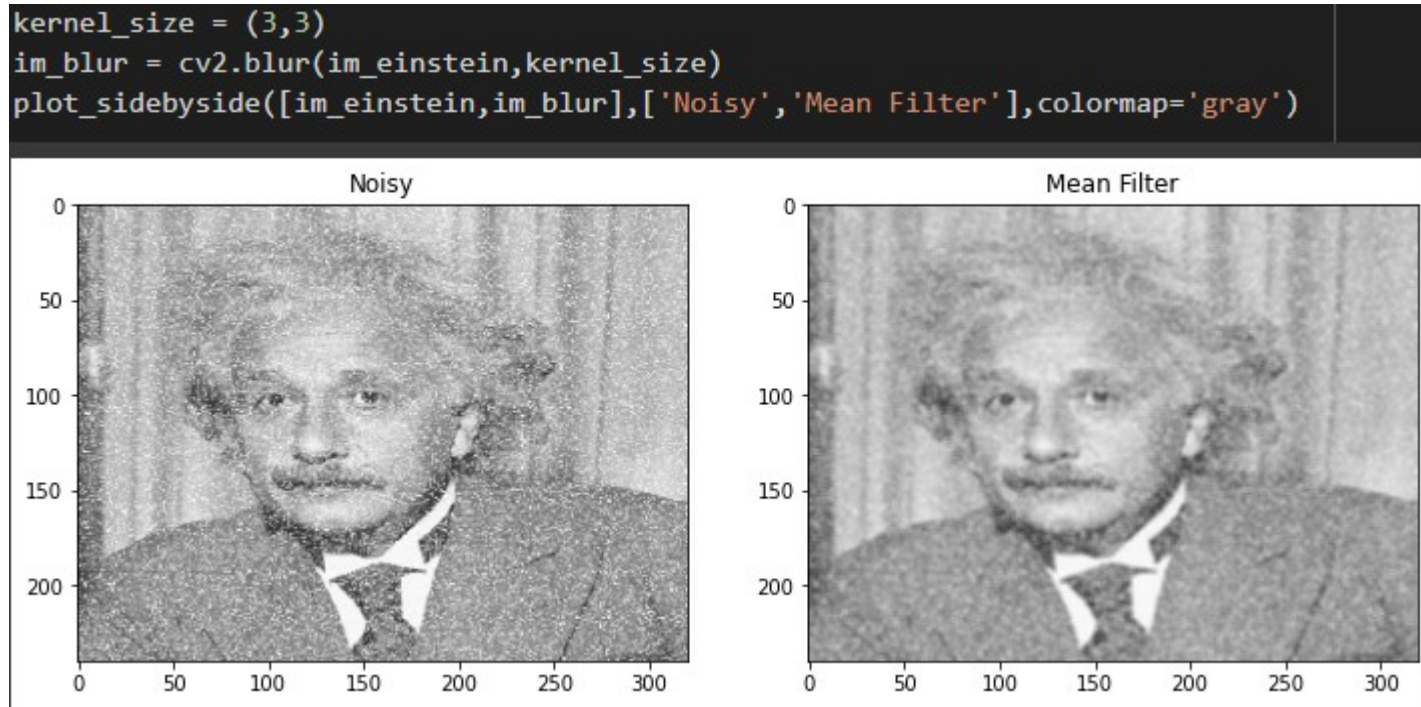
Mean Filter (Low-Pass)

- Replaces the center pixel with the mean of its neighborhood
- Spreads (Average) the outlier value to its neighbors
- Details are smoothed



Mean Filter (Low-Pass)

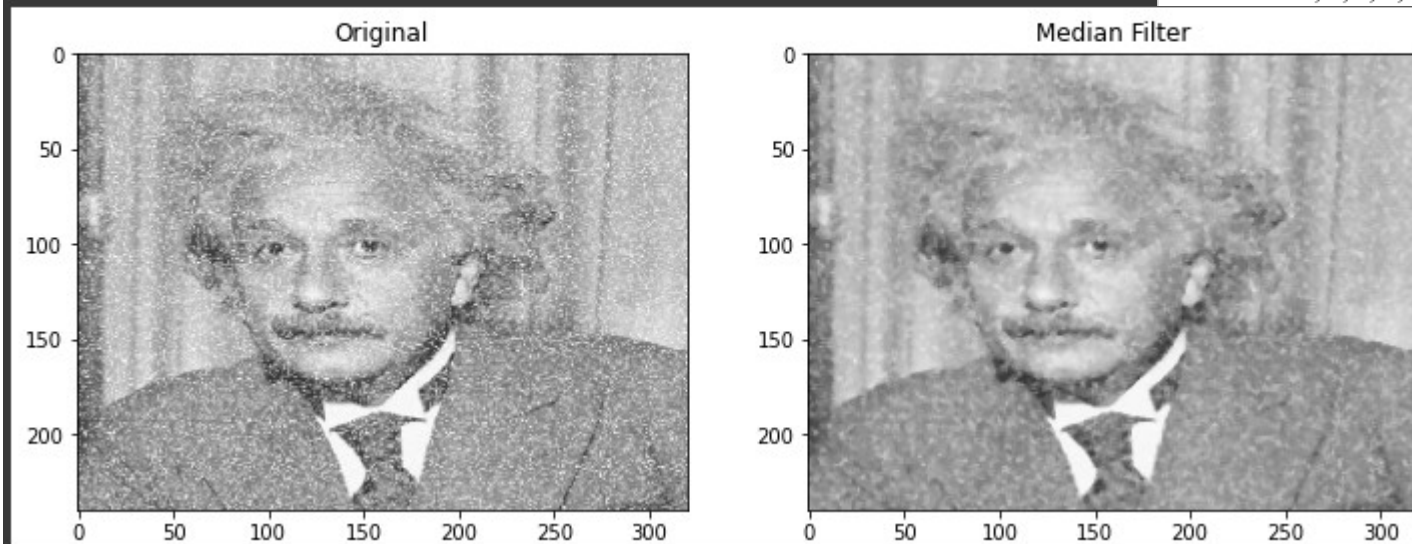
- Replaces the center pixel with the mean of its neighborhood
- Spreads (Average) the outlier value to its neighbors
- Details are smoothed



Median Filter

- Replaces the center pixel with by a median of its neighborhood
- Preserves more details when compared to the mean filter

```
im_blur = cv2.medianBlur(im_einstein,3)
plot_sidebyside([im_einstein,im_blur],['Noisy','Median Filter'],color=
```



Input

1	4	0	1	3	1
2	2	4	2	2	3
1	0	1	0	1	0
1	2	1	0	2	2
2	5	3	1	2	5
1	1	4	2	3	0

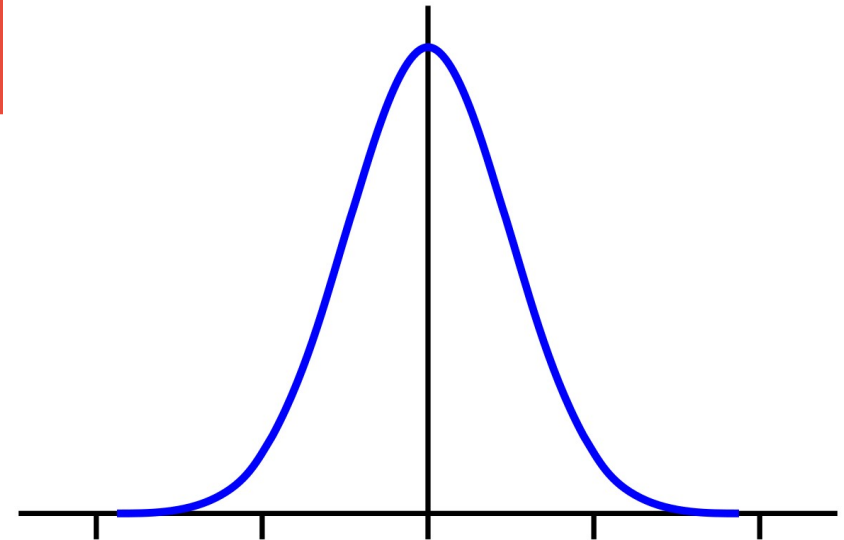
Sorted:0,0,1,1,1,2,2,4,4

Output

1	4	0	1	3	1
2	1	1	1	1	3
1	1	1	1	2	0
1	1	1	1	1	2
2	2	2	2	2	5
1	1	4	2	3	0

Gaussian Filter (Low-Pass)

- Gaussian distribution of pixels
- The kernel is composed of probabilities
- Weighted Mean
- The standard deviation determines the blur degree



$$\frac{1}{16}$$

×

1	2	1
2	4	2
1	2	1

$$\frac{1}{273}$$

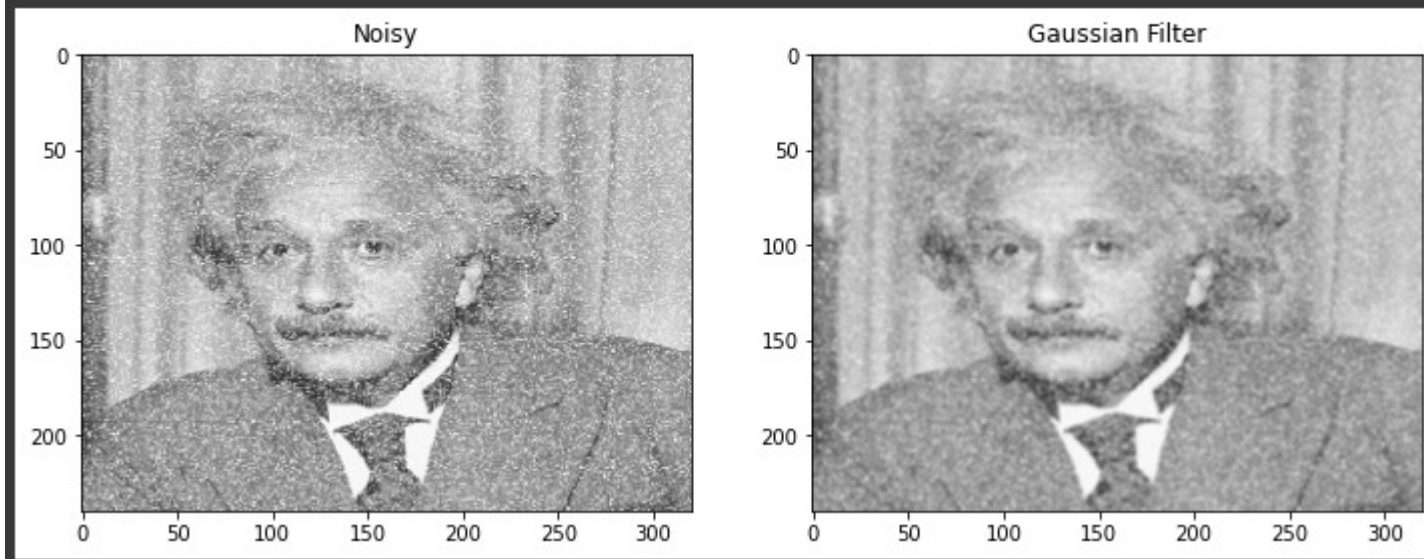
×

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

Gaussian Filter

- Gaussian distribution of pixels
- The kernel is composed of probabilities
- Weighted Mean
- The standard deviation determines the blur degree

```
im_blur = cv2.GaussianBlur(im_einstein,(3,3),0)  
plot_sidebyside([im_einstein,im_blur],['Noisy','Gaussian Filter'],colormap='gray')
```



Bilateral Filter

- Gaussian Distribution based (spacial and pixel intensity)
- Add Normalization Factors and Range Weight
- Preserve details

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(|I_p - I_q|) I_q$$

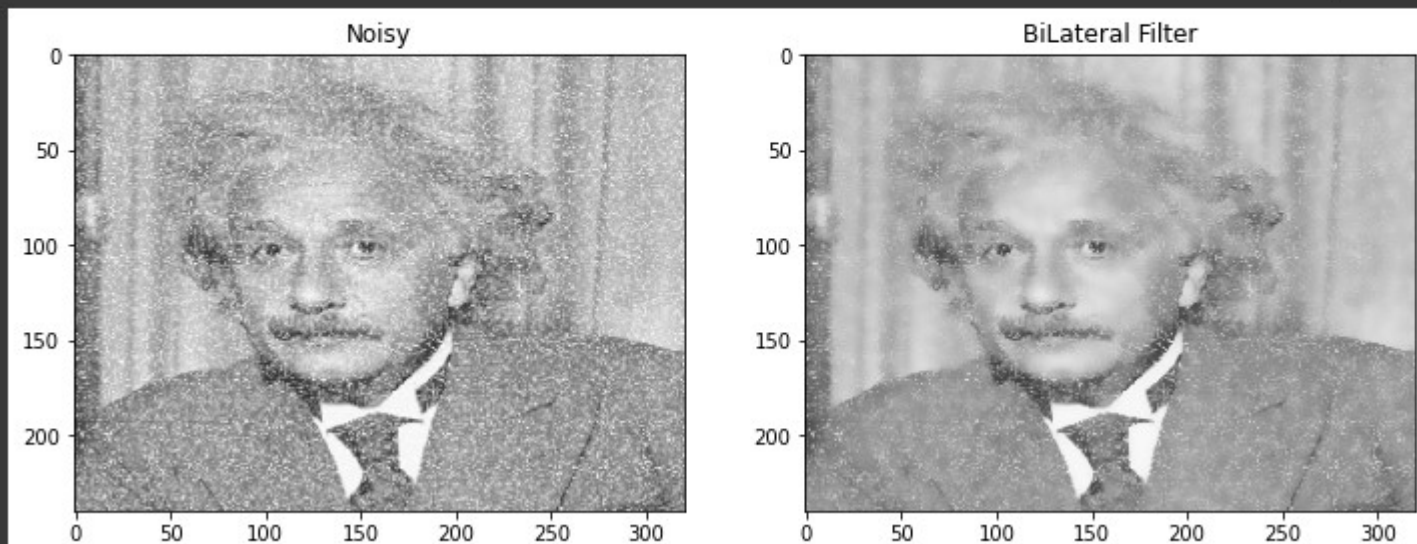
Diagram illustrating the components of the Bilateral Filter equation:

- Normalization Factor:** $\frac{1}{W_p}$
- Space Weight:** $G_{\sigma_s}(\|p - q\|)$ (represented by a 2D Gaussian image)
- Range Weight:** $G_{\sigma_r}(|I_p - I_q|)$ (represented by a 1D Gaussian curve)

Bilateral Filter

- Gaussian Distribution based (spacial and pixel intensity)
- Add Normalization Factors and Range Weight
- Preserve details

```
# Apply bilateral filter with d = 15,  
# sigmaColor = sigmaSpace = 75.  
im_blur = cv2.bilateralFilter(im_einstein,9,75,75)  
plot_sidebyside([im_einstein,im_blur],['Noisy','BiLateral Filter'],colormap='gray')
```



Comparison

Original Image	Noisy Image (Gaussian Noise)	Mean Filter	MEAN FILTER									
			- Replaces each pixel by the average of its neighbors. - Simple and fast. - Blurs edges. 3x3 Kernel (example) <table border="1"> <tr> <td>1/9</td> <td>1/9</td> <td>1/9</td> </tr> <tr> <td>1/9</td> <td>1/9</td> <td>1/9</td> </tr> <tr> <td>1/9</td> <td>1/9</td> <td>1/9</td> </tr> </table>	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9	1/9
1/9	1/9	1/9										
1/9	1/9	1/9										
1/9	1/9	1/9										
			GAUSSIAN FILTER - Weighted average using a Gaussian distribution. - Gives more importance to nearby pixels. - More natural smoothing. 3x3 Gaussian Kernel (example) <table border="1"> <tr> <td>1</td> <td>2</td> <td>1</td> </tr> <tr> <td>2</td> <td>4</td> <td>2</td> </tr> <tr> <td>1</td> <td>2</td> <td>1</td> </tr> </table> 1/16	1	2	1	2	4	2	1	2	1
1	2	1										
2	4	2										
1	2	1										
			MEDIAN FILTER - Replaces each pixel by the median of its neighbors. - Non-linear filter. - Very effective for salt-and-pepper (impulse) noise. 3x3 Window (example) <table border="1"> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>0</td> <td>255</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> </table> Median = 1	1	0	1	0	255	0	1	0	1
1	0	1										
0	255	0										
1	0	1										
			BILATERAL FILTER - Weights depend on both spatial distance and intensity similarity. - Smooths while preserving edges. - More computationally expensive. Spatial closeness (near pixels) Intensity similarity (similar intensities)									

Practice

[Link: Practice 02](#)

Sharpening and Edge Filters

Sharpening and Edge Filters

- Smoothing suppresses high-frequency information; sharpening and edge filters emphasize it.
- High Frequencies are associated with:
- Edges and object boundaries, Fine details, Textures, Noise

Original Image



Edge Detection (Sobel)



Sharpening (Unsharp Masking)



Sharpening and Edge Filters

- Smoothing suppresses high-frequency information; sharpening and edge filters emphasize it.
- High Frequencies are associated with:
- Edges and object boundaries, Fine details, Textures, Noise

Original Image



Edge Detection (Sobel)



Sharpening (Unsharp Masking)



Sharpening

A simple sharpening kernel is:

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

For example, consider this neighborhood:

$$I = \begin{bmatrix} 2 & 2 & 4 \\ 2 & \boxed{5} & 5 \\ 5 & 5 & 5 \end{bmatrix}$$

The new center pixel is:

$$\begin{aligned} (5 \times 5) - (2 + 2 + 5 + 5) \\ = 25 - 14 = \boxed{11} \end{aligned}$$

So:

$$\boxed{5 \rightarrow 11}$$

The center pixel differs from some of its neighbors, and sharpening **amplifies this local intensity difference**.

Original Image



Sharpening (Unsharp Masking)



Sharpening

- Unsharp Mask (blur + sharp)

Mathematically:

$$I_{\text{sharp}} = 1.8I - 0.8I_{\text{blur}}$$

The important observation is:

$$1.8 - 0.8 = 1$$

Therefore, in a **smooth region**, where the original and blurred pixels are approximately equal,

$$I \approx I_{\text{blur}}$$

For example:

$$I = 100, \quad I_{\text{blur}} = 100$$

then:

$$I_{\text{sharp}} = 1.8(100) - 0.8(100) = \boxed{100}$$

So smooth regions remain approximately unchanged.

Now consider a pixel near an edge:

$$I = 150, \quad I_{\text{blur}} = 120$$

Then:

$$I_{\text{sharp}} = 1.8(150) - 0.8(120) = 270 - 96 = \boxed{174}$$

The difference from the neighborhood is **amplified**:

$$150 - 120 = 30$$

becomes:

$$174 - 120 = 54$$

Original Image



Sharpening (Unsharp Masking)



Sharpening

- Unsharp Mask (blur + sharp)

```
blurred = cv2.GaussianBlur(img, (0, 0), sigmaX=2)

sharpened = cv2.addWeighted(
    img,      1.8,      # original image
    blurred, -0.8,      # subtract blurred component
    0
)
```

Original Image



Sharpening (Unsharp Masking)



Edge Detection

- Identifies locations of significant intensity changes
- Produces a representation emphasizing object boundaries

Original Image



Sobel



Prewitt



Laplacian



Edge Detection

1) SOBEL OPERATOR (First Derivative)

Horizontal Gradient (G_x)

-1	0	+1
-2	0	+2
-1	0	+1

Vertical Gradient (G_y)

-1	-2	-1
0	0	0
+1	+2	+1

- Uses larger weights (2) in the center row/column.
- Provides some smoothing $\rightarrow$ less sensitive to noise.
- Better edge localization than Prewitt.

2) PREWITT OPERATOR (First Derivative)

Horizontal Gradient (G_x)

-1	0	+1
-1	0	+1
-1	0	+1

Vertical Gradient (G_y)

-1	-1	-1
0	0	0
+1	+1	+1

- Equal weights (1) in all rows/columns.
- Simpler than Sobel.
- Slightly more sensitive to noise than Sobel.

3) LAPLACIAN OPERATOR (Second Derivative)

Laplacian Kernel (4-neighborhood)

0	+1	0
+1	-4	+1
0	+1	0

- Second derivative (non-directional).
- Responds to rapid intensity changes in all directions.
- More sensitive to noise; edges appear as thin lines

Original Image



Sobel



Prewitt



Laplacian



Practice

[Link: Practice 02](#)